第11-12课时｜MCP协议与Skill生态

MBA课程《大模型智能体》

90分钟　讲授 + 演示 + 实操

- 主题：从“工具能接上”到“工具可治理、可扩展、可复用”
- 目标：理解MCP、会配Claude Desktop、会设计Skill落地方案

你将掌握什么

1. 识别企业工具集成的核心痛点与隐藏成本
2. 理解MCP的协议哲学、角色分层与能力模型
3. 读懂JSON-RPC消息流与错误处理机制
4. 完成Claude Desktop MCP配置与排障
5. 掌握Skill系统设计、发布与治理方法
6. 基于生态现状做选型与路线图

课程地图（11-12课时）

1. 工具集成为什么会失控
2. MCP协议原理与能力边界
3. JSON-RPC架构与消息生命周期
4. Claude Desktop MCP配置实战
5. Skill系统：轻量扩展与组织复用
6. 生态现状与企业落地建议

先问一个业务问题

> 你在公司里推动“AI + 工具自动化”时，最怕哪件事？

- A. 一上生产就不稳定
- B. 权限边界说不清
- C. 维护靠“某位大神”
- D. 工具太多，标准太乱

本节课会逐一对齐这四类问题。

Part 1｜工具集成痛点（为什么需要MCP）

- 先看现实：为什么“能调用工具”并不等于“能规模化”
- 你会看到：
  - 技术复杂度如何指数上升
  - 运维、治理、审计为何成为瓶颈
  - 为什么必须要一个通用协议层

经典困境：M×N 连接器爆炸

- 3个Agent × 20个系统 = 60条集成链路
- 每条链路都要处理：认证、限流、错误、重试、升级
- 任一系统API变更，可能影响多条链路

方案	接入复杂度	维护复杂度
各做各的	M×N	极高
协议标准化	M+N	可控

痛点1：重复造轮子

- 每个团队都在重复做“同类连接器”
- 能力定义不统一：`create_ticket` / `new_issue` / `add_bug`
- 同样功能，不同Agent行为差异大

结果：成本上升，质量下降，知识无法沉淀

痛点2：稳定性不可预期

- API限流、字段变化、权限过期都可能导致调用失败
- 缺少统一重试策略与降级策略
- 失败后难归因：是Agent逻辑、网络，还是工具端问题？

业务视角：自动化链路无法承诺SLA。

痛点3：安全与合规难治理

- 谁可以调用“删除客户数据”？
- 工具调用是否有审计日志?
- 是否能做最小权限、按环境隔离（dev/stage/prod）？

没有标准接口时，安全策略往往散落在脚本里。

痛点4：组织协作成本高

- 平台团队、业务团队、安全团队语言不一致
- “能跑起来”和“可运营”之间有巨大鸿沟
- 缺少统一能力目录（discoverability）

结论：需要像HTTP一样的“工具调用标准层”。

MCP的设计哲学

1. **协议优先**：先统一接口，再谈生态繁荣
2. **能力可发现**：工具能力应可枚举、可理解
3. **模型无关**：不绑定某个LLM厂商
4. **传输可替换**：stdio、本地；HTTP、远程
5. **安全可治理**：权限、审计、边界明确

MCP一句话定义

> MCP（Model Context Protocol）是面向模型应用的上下文与工具交互标准。

- 对标思路：
  - HTTP统一Web通信
  - SQL统一数据库查询
  - MCP统一“模型 ↔️ 外部能力”

MCP解决的不是“调用”，而是“协作”

- **调用层**：让模型能用工具
- **协作层**：让多工具可组合、可治理、可观测
- **组织层**：让能力可以在团队间复用

这是“Demo能跑”到“企业能用”的关键跨越。

管理者翻译页①｜为什么这部分和你有关

- **业务价值**：标准化后，跨部门自动化项目的交付周期通常可从“按月”缩短到“按周”
- **主要风险**：若没有统一协议，后续每新增一个系统都会带来额外集成债务
- **预算影响**：前期会增加协议改造投入（约占PoC预算10%-20%），但可显著降低后续维护成本

Part 2｜MCP协议原理

- 角色分层
- 能力模型（Tools / Resources / Prompts / Sampling）
- 生命周期与消息规范
- 错误、取消、通知、版本协商

MCP三角色：Host / Client / Server

角色	典型实例	职责
Host	Claude Desktop, IDE Agent	管理会话与策略
Client	Host中的MCP客户端	发送JSON-RPC请求
Server	GitHub/DB/Filesystem等服务	暴露标准能力

角色关系图（概念）

```
User
  │
  ▼
Host (Claude Desktop / IDE)
  ├─ MCP Client A ── JSON-RPC ── MCP Server: github
  ├─ MCP Client B ── JSON-RPC ── MCP Server: postgres
  └─ MCP Client C ── JSON-RPC ── MCP Server: filesystem
```

- Host可管理多个Server
- 每个Server独立声明能力

MCP能力模型总览

能力	作用	典型场景
Tools	可执行动作	创建Issue、发消息、写库
Resources	可读取上下文	文件、记录、配置
Prompts	模板化提示	评审模板、任务模板
Sampling	Server请求模型推理	复杂链式决策

Tools：可执行的“函数接口”

- 包含：名称、描述、输入Schema
- Host/模型基于语义决定是否调用
- 返回可结构化结果或文本结果

```
{
  "name": "create_issue",
  "description": "Create a GitHub issue",
  "inputSchema": {
    "type": "object",
    "properties": {
      "repo": {"type": "string"},
      "title": {"type": "string"}
    },
    "required": ["repo", "title"]
  }
}
```

Resources：可读取的“数据入口”

- 以URI抽象外部数据源
- 可用于长文档、结构化数据、状态快照
- 强调“读语义”与“可追踪来源”

```
{
  "uri": "postgres://sales/orders?date=2026-02-01",
  "name": "Orders Snapshot",
  "mimeType": "application/json"
}
```

Prompts：组织知识的复用层

- 把高质量提示词产品化
- 支持参数化模板，减少Prompt漂移
- 适合沉淀：审计、报告、分析、客服流程

```
Prompt: review_pr
Args: repo, pr_number, policy_level
```

Sampling（进阶能力）

- Server可请求Host进行模型推理
- 适合：Server端编排里需要“智能判断”
- 风险：责任边界更复杂，需严格审计

课堂建议：先掌握Tools/Resources，再引入Sampling。

管理者翻译页②｜能力模型的管理含义

- **业务价值**：Tools/Resources/Prompts 分层后，能力复用率更高，跨团队复制更快
- **主要风险**：Sampling等进阶能力若越权调用，会带来责任归属不清与审计难题
- **预算影响**：建议将预算按“70%基础能力 + 20%治理 + 10%创新试验”配置

MCP生命周期（高层）

1. 建立连接（stdio/HTTP）
2. `initialize` 协商能力与版本
3. `initialized` 通知完成
4. `tools/list` / `resources/list` 能力发现
5. `tools/call` 执行与返回
6. 会话结束与资源回收

初始化握手示意

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-06-18",
    "clientInfo": {"name": "claude-desktop", "version": "1.0.0"},
    "capabilities": {"roots": {}, "sampling": {}}
  }
}
```

会话中最常见调用链

```
Host -> tools/list         (          )
Host -> tools/call         (          )
Server -> result/error     (            )
Host ->                   (       /   )
```

重点：模型推理与工具执行是分离的，便于审计。

通知与取消机制

- 通知：无`id`，不要求响应（如能力变更提示）
- 取消：`notifications/cancelled`用于中止耗时操作
- 对长任务必须支持“可取消”，避免系统阻塞

错误处理基本模型

错误层	示例	处理建议
协议错误	非法JSON-RPC	直接拒绝，记录日志
参数错误	缺字段/类型错	返回可读错误 + 修复建议
业务错误	权限不足/资源不存在	提示用户与策略系统
系统错误	网络超时/下游异常	重试 + 降级 + 告警

传输层：stdio vs Streamable HTTP

维度	stdio	Streamable HTTP
部署	本地进程	远程服务
门槛	低	中高
安全	本机边界为主	需鉴权/网关
适用	个人工具、本地研发	团队共享、生产环境

stdio模式深入

- Host拉起Server子进程
- 通过stdin/stdout交换JSON-RPC
- 本地开发体验好，调试直观

```
claude-desktop
  └─ spawn: mcp-server-github
      ├─ stdin  <- request
      └─ stdout -> response
```

HTTP模式深入

- Server作为网络服务提供端点
- 适合集中部署、集中治理
- 常与API网关、身份系统结合

```
Host -> HTTPS -> Gateway -> MCP Server Cluster
```

- 需重点补齐：认证、限流、审计、多租户隔离

安全分层建议（企业）

1. **身份层**：OAuth / API Key / 短期令牌
2. **权限层**：按工具、按参数、按环境授权
3. **执行层**：沙箱、命令白名单、只读默认
4. **审计层**：谁在何时调用了什么，结果如何

可观测性指标（MCP）

- 工具调用成功率（按工具维度）
- P95响应时间 / 超时率
- 参数错误率（提示Schema质量）
- 安全拒绝率（权限策略是否合理）
- 单次任务工具调用次数（成本与稳定性信号）

Part 3｜JSON-RPC架构（工程视角）

- JSON-RPC 2.0是MCP消息骨架
- 理解它，才能真正排障与治理

JSON-RPC 2.0基础

- 核心字段：`jsonrpc`、`id`、`method`、`params`
- 响应字段：`result` 或 `error`
- 通知消息：无 `id`

参考标准：
[JSON-RPC Specification]

Request/Response完整示例

```
// request
{
  "jsonrpc": "2.0",
  "id": "req-42",
  "method": "tools/call",
  "params": {
    "name": "create_issue",
    "arguments": {
      "repo": "org/project",
      "title": "MCP timeout on prod"
    }
  }
}

// response
{
  "jsonrpc": "2.0",
  "id": "req-42",
  "result": {
    "content": [{"type": "text", "text": "Issue #128 created"}]
  }
}
```

错误响应示例

```
{
  "jsonrpc": "2.0",
  "id": "req-42",
  "error": {
    "code": -32001,
    "message": "Permission denied",
    "data": {
      "policy": "github.write.prod",
      "hint": "Request temporary approval"
    }
  }
}
```

- `message` 给人看
- `data` 给系统自动化处理

为什么要重视 `id`

- 并发调用时，靠 `id` 对齐请求与响应
- 便于Tracing：日志、链路、故障回放
- 与业务 `trace_id` 映射，可打通观测平台

建议： `id` 使用可追踪格式，如 `trace-uuid-step`。

批量调用与编排

- JSON-RPC支持批量请求（batch）
- 但课堂实践里，优先“显式串行 + 明确依赖”
- 避免并发调用导致数据竞争或配额突发

```
Step1        -> Step2     -> Step3
```

超时、重试与幂等性

问题	典型风险	工程策略
超时	请求悬挂	统一超时预算
重试	重复写入	幂等键/去重表
下游抖动	雪崩	指数退避 + 熔断

Anthropic: 工具设计的艺术

> 来源: Building Effective Agents - Appendix 2

核心观点： ACI (Agent-Computer Interface) 和 HCI 一样重要

工具设计原则：

1. **站在模型角度思考** — 描述是否足够清晰?
2. **测试模型如何使用** — 用Workbench观察失败模式
3. **Poka-yoke** — 防呆设计，让错误难以发生
4. **给足思考空间** — 避免让模型"写进死角"

工具格式的选择

格式	优点	缺点
Diff	紧凑	需要预知行数
全文重写	简单	token消耗大
Markdown代码块	自然	无需额外转义
JSON内嵌代码	结构化	需转义换行引号

建议：选择模型"在互联网上见过最多"的格式

CLAUDE.md：项目级指令文件

```
# CLAUDE.md

##
                    ...

##
-      TypeScript strict mode
-
-           > 80%

##
-             config/prod.json
- API              mock
```

作用：每次会话开始自动加载，确保Agent了解项目上下文

Schema设计原则（Tools）

1. 字段名语义清晰，避免缩写谜语
2. 必填项最小化，默认值显式化
3. 枚举值可解释（含业务注释）
4. 错误提示可直接指导修复

好Schema vs 坏Schema

维度	好Schema	坏Schema
命名	`customer_id`	`id`
枚举	`priority: low/med/high`	`level: 1/2/3`
描述	业务上下文完整	只有技术术语
校验	类型/范围明确	运行时才报错

验证链路：三层防线

1. **模型前校验**：Prompt约束 + 例子
2. **Server入参校验**：JSON Schema严格校验
3. **业务规则校验**：权限、状态机、配额

原则：错误尽量前置，失败尽量可解释。

管理者翻译页③｜为什么要投入Schema与错误码

- **业务价值**：问题定位时间可从“小时级”降到“分钟级”，降低业务中断成本
- **主要风险**：无统一错误码会导致跨团队扯皮，影响SLA承诺
- **预算影响**：建议在项目早期预留5%-8%预算建设观测与错误治理能力

Part 4｜Claude Desktop MCP配置

- 从“理解协议”到“实际可用”
- 重点：配置、验证、日志、排障、权限

先决条件清单

- 已安装 Claude Desktop
- 可用Node.js / Python运行MCP Server
- 了解本机环境变量与路径
- 准备一个可测试的Server（如filesystem）

Claude Desktop配置文件位置

平台	路径
macOS	`~/Library/Application Support/Claude/claude_desktop_config.json`
Windows	`%APPDATA%\\Claude\\claude_desktop_config.json`

> 修改后通常需要重启Claude Desktop生效。

配置结构（核心字段）

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/Users/you/Documents"],
      "env": {}
    }
  }
}
```

- `mcpServers`：可同时配置多个Server
- 每个Server定义启动命令与参数

多Server配置示例

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/Users/you/work"]
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {"GITHUB_TOKEN": "${GITHUB_TOKEN}"}
    }
  }
}
```

远程Server思路（HTTP）

- Claude Desktop常见是stdio本地模式
- 企业场景可在网关后部署远程MCP服务
- 关键点：统一身份、访问控制、租户隔离

```
Claude Desktop ->        -> MCP       ->
```

配置排障清单（最常见）

1. 命令不存在（`command not found`）
2. 包安装失败（网络或权限）
3. 环境变量未注入（token为空）
4. 路径权限不足（文件系统拒绝）
5. Server启动后立即退出（参数错误）

日志与定位方法

- 先看Claude Desktop日志
- 再在终端独立运行Server，确认是否可启动
- 逐步最小化配置：先filesystem，再加复杂Server
- 对每次失败保留错误快照，形成团队FAQ

权限治理建议（桌面到企业）

层级	建议
个人开发	只启用必要Server，最小目录授权
团队测试	分环境Token，按组分配工具权限
生产环境	网关鉴权 + 审计留痕 + 审批流程

动手试试 1｜安装与验证Claude Desktop

- 平台直达：
  Claude Desktop下载页
- 文档直达：
  Claude Desktop Docs

任务：确认本机可打开Claude Desktop并完成登录。

动手试试 2｜接入一个官方MCP Server

- 平台直达：
  MCP官方Servers仓库
- 推荐起步：`filesystem` 或 `fetch`

任务：选择一个Server，写入配置，重启Claude并尝试调用。

动手试试 3｜用Inspector看消息流

- 平台直达：
  MCP Inspector仓库
- 参考资料：
  MCP官方文档

任务：观察一次`tools/list`和`tools/call`完整消息。

管理者翻译页④｜配置实战后的经营视角

- **业务价值**：完成最小配置后，可在2周内验证一个真实流程的自动化可行性
- **主要风险**：凭证管理与目录权限配置不当，可能导致数据暴露
- **预算影响**：PoC阶段建议单列“安全与权限治理”预算，不低于总预算的15%

Part 5｜Skill系统（OpenClaw视角）

- Skill是“协议之外”的另一条高效扩展路径
- 优势：轻、快、可定制
- 风险：跨平台复用较弱，需要治理规范

Skill是什么

> Skill = 让Agent通过“说明 + 脚本 + 资源”获得稳定能力。

- 常见组成：`SKILL.md`、脚本、配置、模板
- Agent先读说明，再决定调用什么命令
- 特别适合内部流程与垂直场景

Skill与MCP：不是替代，而是分工

维度	MCP Server	Skill
标准化	强（跨Host）	弱（平台相关）
上手速度	中	快
复用范围	广	团队内高
治理复杂度	高	中

Skill目录结构建议

```
my-skill/
├─ SKILL.md          #
├─ scripts/
│  ├─ run.sh
│  └─ validate.py
├─ templates/
│  └─ report.md
└─ config.yaml       #    /
```

SKILL.md应写什么

1. 目标与适用场景
2. 前置依赖与权限要求
3. 输入参数与示例
4. 异常处理与回滚办法
5. 安全边界（禁止操作清单）

原则：让“新同事5分钟可上手”。

一个可执行的SKILL.md片段

```
## 
python scripts/generate_report.py --date {date} --team {team}

## 
- date: YYYY-MM-DD
- team: sales | ops | finance

## 
-                    "retry_after=60"
```

Skill调度机制（简化）

```
User intent
   ↓
Agent       Skill
   ↓
         Skill +
   ↓
       /
   ↓
```

- 成功率高度依赖：描述清晰度 + 参数约束质量

Skill质量检查清单

- 是否有明确“适用/不适用场景”
- 是否提供最小可运行示例
- 错误是否可定位、可恢复
- 是否声明了权限与风险
- 是否有版本号与变更记录

Skill发布与共享

- 平台直达：
  ClawHub
- CLI技能：`clawhub search/install/update/publish`
- 推荐流程：内测 -> 团队试点 -> 全员发布

动手试试 4｜创建你的第一个Skill

- 平台直达：
  OpenClaw Skills文档（示例入口）
- 参考模板：
  ClawHub

任务：做一个“日报生成Skill”，至少包含输入、命令、失败处理。

动手试试 5｜发布并版本化Skill

- 平台直达：
  ClawHub发布指南入口
- 平台直达：
  GitHub Releases

任务：给Skill打`v0.1.0`，记录一次可回滚发布。

管理者翻译页⑤｜Skill 与 MCP 如何组合投资

- **业务价值**：Skill负责快速试错，MCP负责标准化沉淀，组合可兼顾速度与规模
- **主要风险**：若只做Skill不做标准化，半年后可能出现“脚本孤岛”
- **预算影响**：建议采用“双轨预算”：创新试点预算 + 平台化治理预算并行

Part 6｜生态现状与选型策略

- 关注三个维度：生态成熟度、治理能力、组织匹配度
- 技术正确不等于业务可落地

2026 MCP生态观察

1. 工具数量增长很快，但质量参差
2. 通用Server成熟更快（GitHub/DB/Filesystem）
3. 企业更关心：安全、稳定、审计，不只“能不能连”
4. 生态正在从“玩具化”走向“生产化”

数据来源与时间：基于 MCP 官方站点、modelcontextprotocol/servers 仓库公开信息与 Smithery 生态目录抽样统计（访问时间：2026-02-27，口径：公开可访问条目，未去重私有部署实例）。

生态能力地图（简版）

类别	代表Server	典型价值
开发协作	GitHub/GitLab	工程自动化
数据分析	Postgres/BigQuery	报表与决策
办公协同	Slack/Notion	沟通与知识流
本地执行	Filesystem/Shell	自动化运维

国内平台对照（学习用）

方向	平台直达链接	观察点
Agent搭建	Coze	插件生态与发布机制
模型应用	通义百炼	企业接入与成本
流程编排	Dify	工作流 + 工具治理

注：平台能力更新快，课堂后请自行复核最新功能。

选型矩阵：MCP vs Skill vs 混合

业务条件	推荐
跨多AI平台复用	MCP优先
内部流程快速试错	Skill优先
强合规与审计要求	MCP + 网关治理
既要速度又要复用	混合架构

0-1-10落地路线图

1. **0到1（2周）：** 1个关键流程打通
2. **1到N（1-2月）：** 沉淀标准Tool Schema与日志规范
3. **10+流程（季度）：** 平台化治理、权限自动化、成本看板

风险登记（Risk Register）

风险	触发信号	缓解措施
调用失败高	成功率阈值：客服<95%预警、法务<97%预警、交易<99.5%预警	分行业重试策略 + 分级降级 + 人工接管
数据越权	审计告警	最小权限 + 审批
成本失控	调用量激增	配额与预算闸门
人员依赖	只有1人会维护	文档化 + 轮值机制

ROI如何讲给管理层

- 节省重复开发工时（连接器复用）
- 缩短自动化交付周期（上线更快）
- 降低故障定位成本（标准日志）
- 提升合规可解释性（审计闭环）

一句话：从“项目制实验”转为“平台化资产”。

案例蓝图：销售运营自动化

```
CRM     -> MCP Server (postgres)
       -> Agent     -> Tool: create_task / send_summary
       -> Slack     + Dashboard
```

指标	改造前	改造后（8周）	变化
日报汇总耗时	2.5小时/天	0.7小时/天	-72%
线索跟进及时率	61%	86%	+25pct
人工漏跟进工单	34单/周	9单/周	-74%

- 运营每天显著减少人工汇总时间
- 管理层获得可追溯决策链路与周度对比面板

注：以上为课堂案例样本口径（单销售团队、8周观察窗），用于说明前后对比方法。

课堂工作坊（20分钟）

1. 选一个你熟悉的业务流程
2. 划分：哪些能力走MCP，哪些走Skill
3. 设计最小可行架构（角色、权限、日志）
4. 产出1页方案图 + 3条治理策略

讨论题（小组）

- 你的组织最先该标准化哪类工具?
- 你会先做“效率提升”还是“风控治理”？
- 如果只能做一件事，做Schema标准还是日志平台?

建议每组3分钟陈述，1分钟问答。

复盘：六个关键结论

1. 工具集成难点在治理，不在调用
2. MCP把“集成”升级成“标准协作”
3. JSON-RPC是排障与可观测的基础
4. Claude Desktop可作为低门槛实验场
5. Skill适合快速沉淀内部能力
6. 混合架构是多数企业的现实选择

课后作业（建议2小时）

1. 选一个业务流程（如客服质检、销售周报）
2. 输出一份A4方案：
    - 为什么选MCP/Skill/混合
    - Tool Schema草案
    - 权限与审计方案
3. 可选：实现一个最小Demo并录屏5分钟

资料索引（可点击）

官方

- MCP官方文档
- MCP Servers仓库
- MCP Inspector
- JSON-RPC 2.0规范

官方指南 ⭐

- Anthropic: CLAUDE.md & ACI Design - Agent-Computer Interface
- Simon Willison: LLM Tools

平台

- Claude Desktop下载

附录A｜MCP消息字典（速查）

方法	用途
`initialize`	协议能力协商
`tools/list`	枚举工具能力
`tools/call`	执行工具调用
`resources/list`	列出资源
`resources/read`	读取资源内容
`prompts/list`	枚举提示模板

附录B｜最小MCP Server伪代码

```
import { Server } from "@modelcontextprotocol/sdk/server";

const server = new Server({ name: "demo", version: "0.1.0" }, { capabilities: { tools: {} } });

server.setRequestHandler("tools/list", async () => ({
  tools: [{ name: "ping", description: "health check", inputSchema: { type: "object", properties: {} } }]
}));

server.setRequestHandler("tools/call", async (req) => {
  if (req.params.name === "ping") {
    return { content: [{ type: "text", text: "pong" }] };
  }
  throw new Error("unknown tool");
});
```

附录C｜企业上线检查单

- [ ] 关键工具都有Schema与错误码
- [ ] 权限分层可审计
- [ ] 超时/重试/熔断已配置
- [ ] 关键调用有Tracing与告警
- [ ] 发生故障有人工兜底流程

动手试试 6｜生态调研冲刺（课后）

- 平台直达：
  Smithery（MCP生态目录）
- 平台直达：
  MCP官方站点

任务：选3个与你行业最相关的Server，写一页“可用性评估”。

Q&A

从“会调用工具”到“会建设能力平台”

谢谢大家。

下一步建议：把课堂作业直接作为你团队的PoC启动文档。